

Baby Cry Recognition Mode

1. Introduction

The Baby Cry Recognition Mode is one of the core intelligent features of the application. It provides AI-powered audio analysis that detects whether a recorded sound contains a baby cry and, if so, classifies the underlying cause into one of four clinically meaningful categories. The module delivers its results entirely in Arabic, including the cry type name, physical signs to observe, actionable parenting tips, and medical warnings — making it immediately useful for Arabic-speaking caregivers.

This chapter documents the complete technical implementation of the module across two development phases:

- Phase 0 (First Semester):** Focused on building the AI model — selecting the architecture, preparing the dataset, training the two-stage classifier, and creating the initial REST API for inference.
- Phase 1 (Second Semester):** Focused on production hardening — adding audio validation, acoustic sanity checks, rate limiting, structured logging, metrics tracking, and request tracing to make the system robust and deployment-ready.

2. Problem Statement

Infant crying is the primary communication mechanism for neonates and pre-verbal infants. It conveys critical physiological and emotional states — hunger, pain, fear, discomfort, and the need for burping — but decoding these signals remains highly subjective, unreliable, and stressful for caregivers.

2.1 Importance in Healthcare

Dimension	Impact
Neonatal Care	Delayed response to pain cries can indicate undetected medical conditions
Parental Well-being	Inability to soothe a crying infant is a leading contributor to postpartum stress and anxiety
Clinical Settings	NICU nurses monitor multiple infants; automated cry triage could prioritize urgent cases
Accessibility	Hearing-impaired or novice caregivers lack the auditory experience to differentiate cry types

2.2 Challenges in Interpreting Infant Cries

- Acoustic Ambiguity:** Cry signals are non-stationary, overlap in spectral features across categories, and are highly variable between infants.
- Environmental Noise:** Real-world recordings contain background noise, multiple sound sources, and variable microphone characteristics.
- Class Imbalance:** Certain cry types (e.g., burping) are significantly rarer than others (e.g., general needs), creating skewed training distributions.
- Subjectivity of Labels:** Human annotation of cry types is inherently subjective — inter-rater agreement in cry labeling studies is typically moderate.
- Short Duration:** Cry episodes can be very brief (less than 2 seconds), demanding models that can extract meaningful features from limited temporal context.

2.3 Motivation Behind AI-Based Classification

Traditional rule-based acoustic analysis (pitch tracking, formant analysis) has shown limited discriminative power for cry classification. Self-supervised speech representation models, pre-trained on thousands of hours of human speech, capture rich hierarchical acoustic features that transfer effectively to non-speech audio domains. This module leverages that insight by applying HuBERT — a state-of-the-art speech transformer originally trained on 960 hours of English speech — to the pediatric cry domain.

3. Objectives

3.1 Primary Objective

Develop and deploy an end-to-end AI system that automatically detects infant cries from arbitrary audio input and classifies the underlying cause into actionable categories, delivered through a production-grade REST API with full Arabic localization.

3.2 Detailed Objectives

Objective	Description
Hierarchical Classification	Implement a two-stage cascade to separate detection (high-precision binary) from fine-grained classification (multi-class)
Ensemble Robustness	Use multi-checkpoint ensemble averaging to reduce variance in Stage 2 predictions

Objective	Description
Arabic Localization	Develop all user-facing content — cry type names, physical signs, parenting tips, and medical disclaimers — in Arabic
API-First Design	Expose the inference pipeline through a documented FastAPI backend with auto-generated Swagger/OpenAPI documentation
Production Hardening	Add input validation, acoustic checks, rate limiting, structured logging, and metrics for deployment readiness

3.3 Performance Goals

Goal	Target
Binary cry detection accuracy (Stage 1)	≥ 99%
4-class classification accuracy (Stage 2)	≥ 80%
GPU inference latency	< 200 ms
CPU inference latency	< 3 seconds
Supported audio formats	WAV, MP3, OGG, FLAC, M4A, WebM

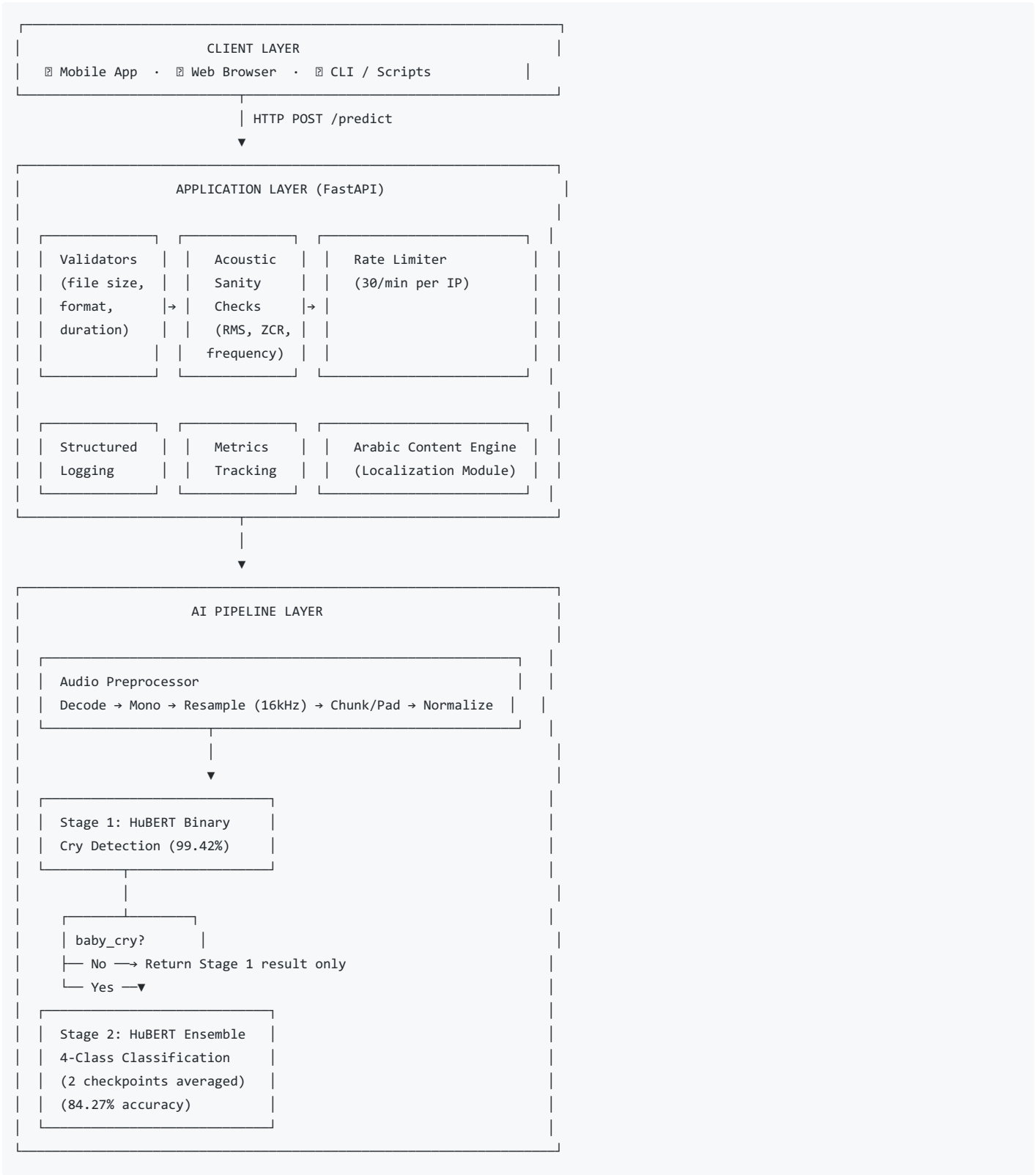
4. System Overview

4.1 High-Level Architecture

The module follows a two-stage hierarchical cascade architecture:

- Stage 1 — Cry Detection:** A binary HuBERT classifier determines whether the input audio contains a baby cry or not (baby_cry vs. not_baby_cry).
- Stage 2 — Cry Type Classification:** If a cry is detected, an ensemble of two HuBERT classifiers identifies the cry type from four categories: scared, needs, physical_pain, or burping.

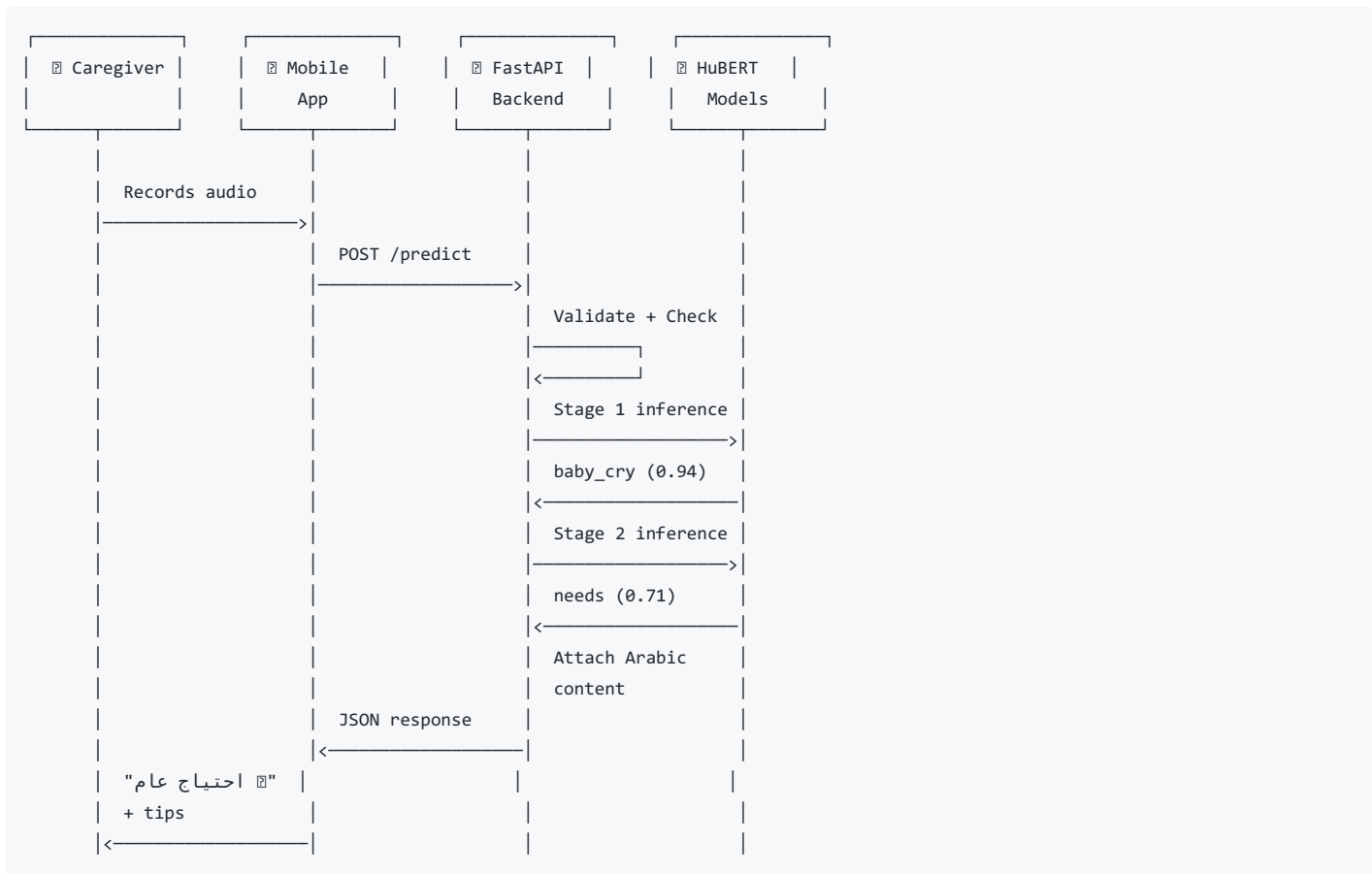
The following diagram illustrates the complete end-to-end pipeline:



4.2 Data Flow Summary

1. **Audio Ingestion:** The client uploads an audio file via `POST /predict` (multipart/form-data).
2. **Validation:** The file is checked for size limits (1 KB – 25 MB), format (WAV/MP3/OGG/FLAC/M4A/WebM), and duration (0.5 – 30 seconds).
3. **Acoustic Sanity Check:** The audio signal is analyzed for RMS energy, voiced frame ratio, frequency band distribution, spectral flux, and zero-crossing rate – rejecting obvious non-cry inputs (silence, noise, constant tones) before inference.
4. **Decoding:** `soundfile.read()` decodes the audio to raw float32 PCM.
5. **Preprocessing:** Stereo-to-mono conversion, resampling to 16 kHz, center-cropping or zero-padding to 80,000 samples (5 seconds), and zero-mean unit-variance normalization.
6. **Stage 1 Inference:** Binary HuBERT classifier produces cry/not-cry probability.
7. **Conditional Stage 2:** If a cry is detected, an ensemble of 2 Stage 2 HuBERT classifiers produces an averaged 4-class probability distribution.
8. **Response Assembly:** Raw predictions are enriched with Arabic content (titles, physical signs, tips, warnings), confidence labels, and a medical disclaimer.
9. **JSON Delivery:** Structured Pydantic response returned to the client with a unique request ID.

4.3 User Interaction Flow



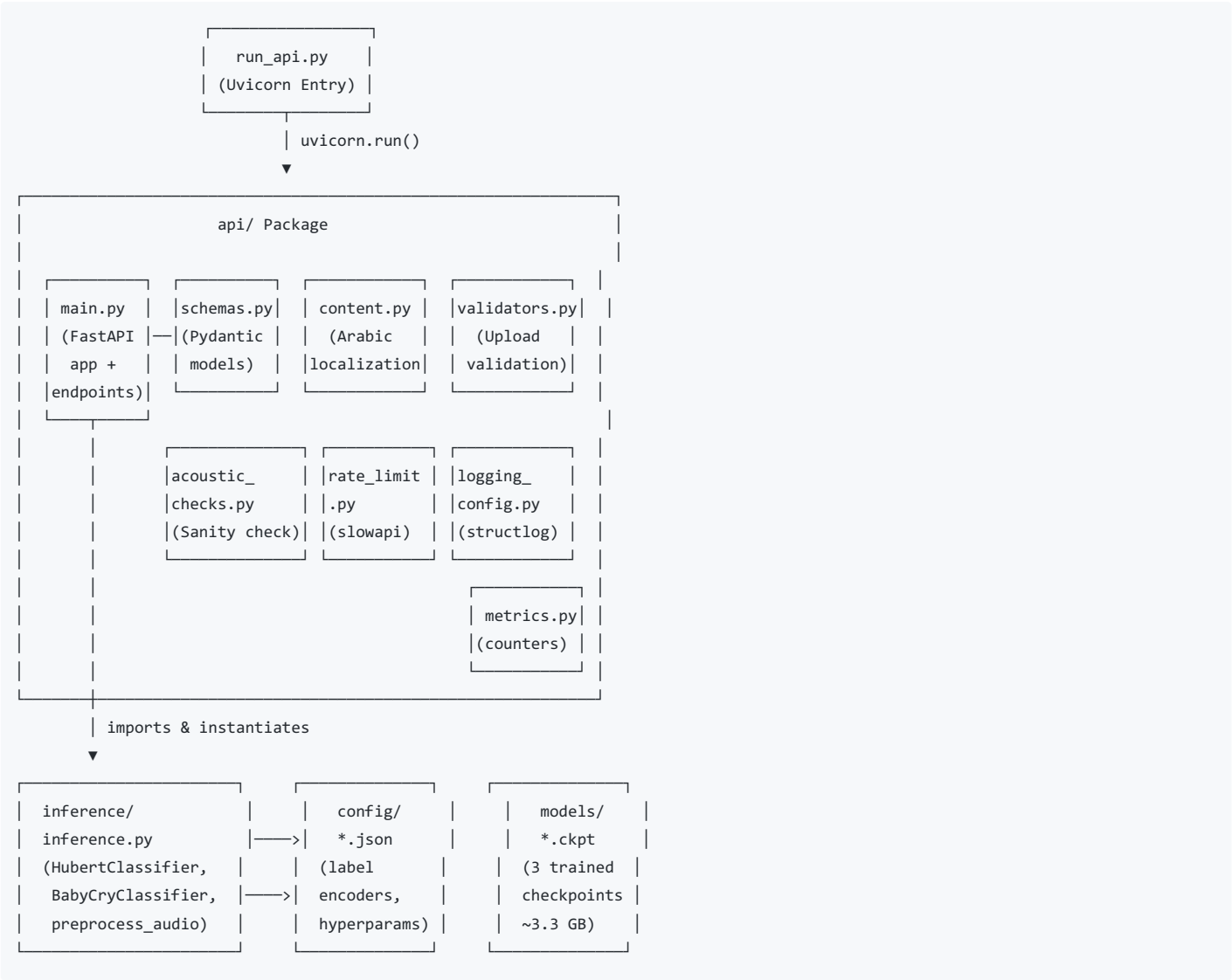
5. Project Structure

5.1 Directory Layout

```
BabyCry/
├── api/                                # FastAPI REST backend
│   ├── __init__.py                    # Package marker
│   ├── main.py                        # Application entry point, endpoints, model lifecycle
│   ├── schemas.py                     # Pydantic v2 response models
│   ├── content.py                     # Arabic localization content & confidence helpers
│   ├── validators.py                  # [NEW] Audio validation (size, format, duration)
│   ├── acoustic_checks.py             # [NEW] Acoustic sanity checks (RMS, ZCR, frequency)
│   ├── rate_limit.py                  # [NEW] Rate limiting (slowapi, 30/min per IP)
│   ├── logging_config.py              # [NEW] Structured logging (structlog, JSON/console)
│   └── metrics.py                     # [NEW] In-memory metrics tracking
├── config/                             # Model configuration & label encoders
│   ├── training_config.json           # Hyperparameters, class weights, label mappings
│   ├── stage1_label_encoder.json      # Binary label encoder (baby_cry / not_baby_cry)
│   └── stage2_label_encoder.json      # 4-class label encoder with merge map
├── docs/                               # Evaluation artifacts
│   ├── stage1_test_results.json       # Stage 1 test metrics
│   ├── stage2_test_results.json       # Stage 2 test metrics
│   ├── stage1_confusion_matrix.png    # Stage 1 confusion matrix visualization
│   └── stage2_confusion_matrix.png    # Stage 2 confusion matrix visualization
├── inference/                          # Core AI inference module
│   └── inference.py                   # HubertClassifier, BabyCryClassifier, preprocess_audio
├── models/                             # Trained model checkpoints (~3.3 GB total)
│   ├── stage1_binary.ckpt             # Stage 1 binary model (~1.05 GB)
│   ├── stage2_4class_epoch6.ckpt      # Stage 2 ensemble member 1 (~1.05 GB)
│   └── stage2_4class_epoch7.ckpt      # Stage 2 ensemble member 2 (~1.05 GB)
├── requirements.txt                    # Python dependency specification
└── run_api.py                         # Uvicorn server launcher (dev/production modes)
```

Items marked with **[NEW]** were added in the second semester as part of the production hardening phase.

5.2 Module Relationships



6. AI/ML Technical Analysis

6.1 Model Architecture

6.1.1 Foundation: HuBERT (Hidden-Unit BERT)

The system is built on **facebook/hubert-base-ls960**, a self-supervised speech representation model pre-trained by Facebook AI Research. HuBERT learns acoustic representations by predicting discretized speech units (cluster IDs from k-means) for masked audio frames — similar to how BERT learns language by predicting masked words.

Component	Specification
Pre-training Data	LibriSpeech 960 hours (English speech)
Architecture	Transformer encoder (12 layers)
Hidden Size	768
Attention Heads	12
Parameters	~95 million
Pre-training Objective	Masked prediction of discretized speech units (k-means cluster IDs)

Feature Extractor Component	7-layer CNN stack (16 kHz raw waveform → frame-level features)
-----------------------------	--

6.1.2 Why HuBERT Over Traditional Approaches

Approach	Limitation	HuBERT Advantage
MFCC + SVM/Random Forest	Handcrafted features miss complex temporal patterns	Learns hierarchical representations end-to-end
Mel Spectrogram + CNN	Requires spectrogram computation; limited temporal modeling	Operates directly on raw waveform
Custom CNN/RNN	Requires large labeled datasets	Self-supervised pre-training leverages 960h unlabeled speech
OpenL3 / VGGish	Pre-trained on environmental sounds; less speech-focused	HuBERT captures fine-grained vocal characteristics

6.1.3 Classification Head Architecture

On top of the HuBERT backbone, a lightweight classification head is added:

```
HuBERT Backbone (frozen CNN feature extractor + fine-tuned transformer)
├─ Masked Mean Pooling (sequence → single vector, dimension 768)
│   └─ Linear(768 → 256) + GELU activation
│       └─ Dropout(0.1)
│           └─ Linear(256 → num_classes)
```

The classification head uses:

- **Masked Mean Pooling:** Averages only non-padded hidden states, ensuring variable-length audio is handled correctly without the padded regions influencing the output.
- **GELU Activation:** Gaussian Error Linear Unit — smoother gradient flow compared to ReLU, standard in transformer architectures.
- **Dropout Regularization:** 10% dropout to prevent overfitting the relatively small cry dataset.

6.1.4 Transfer Learning Strategy

The training employs a discriminative learning rate strategy:

Component	Learning Rate	Strategy
CNN Feature Extractor	Frozen (0)	Fully locked — pre-trained waveform-to-frame conversion
Transformer Layers	1e-5	Fine-tuned with 100× smaller rate than head
Classification Head	1e-3	Trained from scratch

This approach follows the gradual unfreezing paradigm where lower layers (closer to raw input) retain pre-trained knowledge while upper layers adapt to the target domain.

6.2 Audio Processing Pipeline

6.2.1 Preprocessing Steps

The `preprocess_audio()` function in `inference/inference.py` implements a universal audio preprocessing pipeline:



Step	Implementation	Rationale
Decoding	<code>soundfile.read(dtype='float32')</code>	Fast C-based decoder; supports WAV, FLAC, OGG
Mono Conversion	<code>wav.mean(axis=-1)</code>	HuBERT expects single-channel input
Resampling	<code>librosa.resample(orig_sr, target_sr=16000)</code>	HuBERT was pre-trained at 16 kHz
Chunking	Center-crop to 80,000 samples (5 seconds)	Balances context length with memory usage
Padding	Zero-pad + binary attention mask	Handles audio shorter than 5 seconds without information loss
Normalization	$(x - \mu) / \sigma$ per-sample	Matches HuBERT's expected input distribution
int16 Detection	Auto-scales if <code>max > 1.5</code>	Handles integer-encoded audio gracefully

6.2.2 Attention Mask Downsampling

A critical engineering detail: the attention mask (which marks real vs. padded samples at 16 kHz) must be downsampled to match the transformer's internal frame rate. This is handled by the `_downsample_mask()` method using `F.adaptive_max_pool1d()` — a computationally efficient approach that preserves mask semantics (any frame containing at least one real sample is marked as valid).

6.3 Training Pipeline

6.3.1 Dataset Preparation

The training data was aggregated from multiple publicly available sources:

Source	Type	License	Role
baby_crying (Kaggle)	Infant cry recordings	See Kaggle source	Primary cry data (dominant source)
infant_cry_corpus (Donate-a-Cry)	Annotated cry recordings	CC BY-NC 4.0	Secondary cry data with type labels
baby_sounds (Kaggle)	Mixed baby vocalizations	See Kaggle source	Additional cry and non-cry samples
ESC-50	Environmental sounds	CC BY-NC 4.0	Negative class for Stage 1

6.3.2 Label Merging Strategy

The original 6-class taxonomy was consolidated into 4 clinically meaningful categories to improve model robustness:



Rationale: hungry , tired , and discomfort are acoustically similar and share a common caregiving response (check basic needs). Merging improves model robustness at the cost of granularity.

6.3.3 Class Distribution

Class	Test Samples	Proportion	Imbalance Status
needs	626	63.1%	Heavy majority
physical_pain	162	16.3%	Moderate
scared	135	13.6%	Moderate
burping	69	7.0%	Severe minority

6.3.4 Training Configuration

Hyperparameter	Value
Sample Rate	16,000 Hz
Chunk Duration	5.0 seconds (80,000 samples)
Label Smoothing	0.1
Head Hidden Dimension	256
Dropout	0.1
Head Learning Rate	1e-3
Backbone Learning Rate	1e-5
Weight Decay	1e-4
Warmup Steps	500
Framework	PyTorch Lightning 2.4

6.3.5 Class Weights (Inverse Frequency Weighting)

Class	Weight	Interpretation
burping	2.075	Heavily upweighted — severe minority class (~240 samples)
needs	0.203	Heavily downweighted — dominant majority class
physical_pain	0.811	Slightly downweighted

scared Class	0.912 Weight	Near-neutral Interpretation
-----------------	-----------------	--------------------------------

This inverse-frequency weighting scheme compensates for the extreme class imbalance, ensuring the loss function does not ignore minority classes.

6.3.6 Loss Function

Cross-Entropy with Label Smoothing ($\epsilon=0.1$) and class weights. Label smoothing prevents overconfident predictions and improves calibration – critical for a healthcare application where confidence scores are surfaced to users.

6.3.7 Optimizer

Dual learning rate optimization:

- **Classification Head:** Adam with `lr=1e-3`, `weight_decay=1e-4`
- **HuBERT Backbone:** Adam with `lr=1e-5`, `weight_decay=1e-4`
- **Warmup Schedule:** 500 steps linear warmup

6.4 Inference Pipeline

The inference pipeline implements the two-stage cascade with conditional execution:



Key Implementation Details:

1. **@torch.no_grad() decorator:** Disables gradient computation during inference, reducing memory usage by approximately 50% and improving speed.
2. **Ensemble Averaging:** Stage 2 uses probability averaging (not logit averaging), producing smoother confidence distributions.
3. **Cascade Short-Circuit:** If Stage 1 predicts `not_baby_cry`, Stage 2 is entirely skipped – saving approximately 50% inference time for non-cry audio.
4. **Confidence Rounding:** All probabilities are rounded to 4 decimal places for JSON serialization stability.

7. Production Hardening (Second Semester)

This section documents the production hardening improvements implemented during the second semester. These changes transform the prototype from Phase 0 into a deployment-ready system.

7.1 Overview of Improvements

Feature	File	Description
Audio Validation	api/validators.py	File size, format, and duration checks before processing
Acoustic Sanity Checks	api/acoustic_checks.py	Signal-level analysis to reject silence, noise, and tones
Rate Limiting	api/rate_limit.py	Per-IP request throttling (30/min, 500/hour)
Structured Logging	api/logging_config.py	JSON logging in production, colored console in development
Metrics Tracking	api/metrics.py	In-memory counters exposed via /health endpoint
Request ID Tracking	api/main.py	Unique UUID per request for traceability
Model Warm-up	api/main.py	Dummy inference on startup to JIT-compile and warm caches
Enhanced Schemas	api/schemas.py	AudioInfo, acoustic_check, request_id in responses

7.2 Audio Validation (validators.py)

The audio validation module runs **before** any audio processing or inference, catching bad inputs early and providing clear bilingual error messages.

Validation Rules

Check	Limit	Error Code
Filename present	Must have filename	MISSING_FILENAME
File extension	wav, mp3, ogg, flac, m4a, webm	UNSUPPORTED_FORMAT
Minimum file size	1 KB	FILE_TOO_SMALL
Maximum file size	25 MB	FILE_TOO_LARGE
Audio parseable	Must be decodable by SoundFile	INVALID_AUDIO_FORMAT
Minimum duration	0.5 seconds	AUDIO_TOO_SHORT
Maximum duration	30 seconds	AUDIO_TOO_LONG

Implementation

The `validate_upload()` function is an async function that reads the uploaded file bytes and performs all checks sequentially. If any check fails, it raises an `HTTPException` with a structured error containing both Arabic and English messages.

The `get_audio_info()` function extracts metadata (duration, sample rate, channels, format, size) from the validated audio bytes for inclusion in the API response.

7.3 Acoustic Sanity Checks (acoustic_checks.py)

The acoustic sanity check module adds a layer of signal-level validation **after** the file is decoded but **before** the HuBERT inference. This prevents the deep model from wasting compute on inputs that are obviously not infant cries.

Checks Performed

Check	Metric	Threshold	Rejects
Volume	RMS Energy	≥ 0.010 (~-40 dBFS)	Silent or near-silent recordings
Voiced Ratio	Fraction of loud frames	$\geq 20\%$	Mostly-silent recordings
Frequency Band	Energy in 250–3000 Hz band	$\geq 40\%$ of total energy	Non-vocal sounds (e.g., machinery, very low/high frequency noise)

Check	Metric	Threshold	Rejects
Spectral Flux	Onset strength (temporal variation)	≥ 0.015	Constant tones, hums, drones
Zero-Crossing Rate (low)	Average ZCR	≥ 0.02	Unnaturally smooth signals
Zero-Crossing Rate (high)	Average ZCR	≤ 0.35	White noise, harsh static

How It Works

The function loads the audio, converts it to mono at 16 kHz, and then computes five acoustic features in sequence. If any feature falls outside the expected range for a cry-like signal, the function returns immediately with a failure result containing the machine-readable error code, Arabic/English explanations, and the raw measurements for debugging.

All measurements are included in the API response (in the `acoustic_check` field) even when the check passes, providing transparency into the signal characteristics.

7.4 Rate Limiting (rate_limit.py)

Rate limiting prevents abuse of the API by throttling requests per client IP address.

Limit	Window
30 requests	Per minute
500 requests	Per hour

The module uses the `slowapi` library (a wrapper around `limits`), which identifies clients by their IP address via `get_remote_address()`. When a client exceeds the limit, a `429 Too Many Requests` response is returned with bilingual error messages and a `Retry-After: 60` header.

7.5 Structured Logging (logging_config.py)

The logging module configures `structlog` for structured, machine-parseable logging:

Environment	Output Format	Use Case
development	Pretty colored console output	Local debugging
production	JSON (one line per log event)	Log aggregation (Datadog, CloudWatch, ELK)

Log events include structured key-value pairs such as `request_id`, `client_ip`, `stage1_label`, `stage2_label`, `processing_ms`, and `inference_ms` — enabling efficient searching and dashboarding.

7.6 Metrics Tracking (metrics.py)

The metrics module maintains thread-safe, in-memory counters that are exposed through the `/health` endpoint:

Metric	Description
uptime_seconds	Time since server started
requests_total	Total number of prediction requests
predictions_success	Successful predictions
predictions_failed	Failed predictions
validation_errors	Requests rejected by input validation
acoustic_rejections	Requests rejected by acoustic sanity checks
inference_errors	Errors during model inference

rate_limit_hits Metric	Rate limit violations Description
avg_inference_ms	Average inference time in milliseconds
class_predictions	Per-class prediction counts (e.g., {"needs": 42, "scared": 15})

7.7 Request ID Tracking

Every prediction request is assigned a unique UUID (`request_id`). This ID is:

- Generated at the start of request processing
- Included in all log events for that request
- Returned in the API response (both success and error)
- Available for the client to reference when reporting issues

This enables end-to-end request tracing from client to server logs.

7.8 Model Warm-up

On server startup, after loading all three model checkpoints, a dummy inference is run with 5 seconds of silence:

```
dummy_audio = np.zeros(80000, dtype=np.float32)
classifier.predict((16000, dummy_audio))
```

This serves two purposes:

1. **JIT Compilation:** PyTorch lazily compiles CUDA kernels on first use. The warm-up triggers this compilation during startup rather than on the first real request.
2. **Cache Warming:** GPU memory allocation and tensor operation caches are primed, ensuring consistent latency from the very first request.

8. API Documentation

8.1 Base Configuration

Property	Value
Base URL	<code>http://127.0.0.1:8000</code>
Interactive Docs	<code>http://127.0.0.1:8000/docs</code> (Swagger UI)
Alternative Docs	<code>http://127.0.0.1:8000/redoc</code> (ReDoc)
Rate Limit	30 requests/minute per IP

8.2 Endpoints

Method	Path	Description
GET	<code>/</code>	Arabic landing page (HTML)
GET	<code>/health</code>	Health check with live metrics
POST	<code>/predict</code>	Main inference endpoint
GET	<code>/docs</code>	Auto-generated Swagger/OpenAPI
GET	<code>/redoc</code>	Alternative ReDoc documentation

8.3 GET `/health` — Health Check

Returns the server status, model loading state, device, and live operational metrics.

Response Example:

```
{
  "status": "ok",
  "models_loaded": true,
  "device": "cuda",
  "version": "1.0.0",
  "metrics": {
    "uptime_seconds": 3600,
    "requests_total": 150,
    "predictions_success": 142,
    "predictions_failed": 0,
    "validation_errors": 5,
    "acoustic_rejections": 3,
    "inference_errors": 0,
    "rate_limit_hits": 0,
    "avg_inference_ms": 148.5,
    "class_predictions": {
      "needs": 85,
      "scared": 30,
      "physical_pain": 20,
      "burping": 7,
      "not_baby_cry": 8
    }
  }
}
```

8.4 POST /predict — Classify Audio

Main inference endpoint. Accepts an audio file, runs validation, acoustic checks, and the two-stage cascade, returning Arabic-localized results.

Request:

Parameter	Type	Location	Required	Description
audio	file	multipart/form-data	Yes	Audio file (WAV, MP3, OGG, FLAC, M4A, WebM)

cURL Example:

```
curl -X POST http://127.0.0.1:8000/predict \
-F "audio=@baby_crying.wav"
```

Python Example:

```
import requests

with open("baby_crying.wav", "rb") as f:
    response = requests.post(
        "http://127.0.0.1:8000/predict",
        files={"audio": ("baby.wav", f, "audio/wav")}
    )
print(response.json())
```

Success Response — Cry Detected (200 OK)

```

{
  "success": true,
  "request_id": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
  "audio_info": {
    "duration_seconds": 3.2,
    "sample_rate": 44100,
    "channels": 1,
    "format": "WAV",
    "size_kb": 282.5
  },
  "acoustic_check": {
    "rms": 0.0832,
    "rms_db": -21.6,
    "voiced_ratio": 0.75,
    "cry_band_ratio": 0.68,
    "spectral_flux": 0.0452,
    "zcr": 0.089
  },
  "stage1": {
    "is_baby_cry": true,
    "emoji": "👶",
    "title_ar": "تم اكتشاف بكاء الطفل",
    "description_ar": ".نعم، الصوت يحتوي على بكاء طفل. سنحلل الآن نوع البكاء",
    "confidence": 0.94,
    "confidence_label_ar": "ثقة عالية جدا"
  },
  "stage2": {
    "type": "needs",
    "emoji": "👶",
    "name_ar": "احتياج عام",
    "definition_ar": "...بكاء عام يدل على حاجة أساسية: جوع، تعب، أو عدم راحة",
    "physical_signs_ar": [
      "جوع: عادة بعد 2-4 ساعات من الرضعة الأخيرة 🍼",
      "تعب: عيون ثقيلة، تثاؤب، حركات بطيئة 😴",
      "حرارة الجو غير مناسبة 🌡️",
      "حفاضة مبللة 💩"
    ],
    "tips_ar": [
      "افحصي الحفاضة أولاً 🍼",
      "قدمي الرضعة إن مَرَّ وقت كاف 🍼",
      "(تأكدي من حرارة الغرفة (22-24°م مثالية)",
      "حظنيه واهدنيه إن كان متعباً 🤱"
    ],
    "warning_ar": "...راقبي إشارات طفلك قبل البكاء",
    "confidence": 0.71,
    "confidence_label_ar": "ثقة عالية",
    "confidence_advice_ar": "",
    "all_probabilities": {
      "burping": 0.05,
      "needs": 0.71,
      "physical_pain": 0.18,
      "scared": 0.06
    }
  },
  "medical_disclaimer_ar": "...تنبيه مهم: هذا التطبيق يستخدم الذكاء الاصطناعي 🤖",
  "processing_time_ms": 152
}

```

Success Response — No Cry Detected (200 OK)

```
{
  "success": true,
  "request_id": "f8e7d6c5-b4a3-2109-fedc-ba9876543210",
  "audio_info": {
    "duration_seconds": 4.5,
    "sample_rate": 16000,
    "channels": 1,
    "format": "WAV",
    "size_kb": 144.0
  },
  "acoustic_check": {
    "rms": 0.0521,
    "rms_db": -25.7,
    "voiced_ratio": 0.62,
    "cry_band_ratio": 0.55,
    "spectral_flux": 0.032,
    "zcr": 0.12
  },
  "stage1": {
    "is_baby_cry": false,
    "emoji": "👶",
    "title_ar": "لا يوجد بكاء طفل",
    "description_ar": "...الصوت المسجل لا يحتوي على بكاء طفل",
    "confidence": 0.97,
    "confidence_label_ar": "ثقة عالية جدا"
  },
  "stage2": null,
  "medical_disclaimer_ar": "...: تنبيه مهم 🚨",
  "processing_time_ms": 89
}
```

8.5 Error Responses

Code	Error Code	Scenario
400	MISSING_FILENAME	Upload has no filename
400	UNSUPPORTED_FORMAT	File extension not supported
400	FILE_TOO_SMALL	File smaller than 1 KB
400	AUDIO_TOO_SHORT	Audio shorter than 0.5 seconds
400	AUDIO_TOO_LONG	Audio longer than 30 seconds
400	INVALID_AUDIO_FORMAT	File cannot be decoded as audio
400	DECODE_ERROR	Audio decode failure
400	TOO_QUIET	Audio is silent or near-silent
400	MOSTLY_SILENCE	Less than 20% voiced frames
400	WRONG_FREQUENCY_PROFILE	Energy outside expected cry band
400	CONSTANT_TONE	Audio has no temporal variation
400	TOO_SMOOTH	ZCR too low for vocal sounds
400	TOO_NOISY	ZCR too high (white noise)
413	FILE_TOO_LARGE	File larger than 25 MB

429 Code	RATE_LIMIT_EXCEEDED Error Code	Too many requests Scenario
500	INFERENCE_ERROR	Model inference failure
503	MODEL_NOT_LOADED	Models still loading at startup

All error responses include bilingual messages (`error_message_ar` and `error_message_en`), the `request_id` , and `success: false` .

9. Technologies Used

9.1 Core AI/ML Stack

Technology	Version	Purpose
PyTorch	≥ 2.0.0	Deep learning framework with CUDA GPU support
PyTorch Lightning	2.4.0	Structured training loops, checkpoint management
Hugging Face Transformers	4.44.2	Access to HuBERT pre-trained model
TorchMetrics	1.4.2	GPU-accelerated evaluation metrics

9.2 Audio Processing

Technology	Version	Purpose
Librosa	≥ 0.10.0	Audio resampling and acoustic feature extraction
SoundFile	≥ 0.12.1	Fast C-based audio I/O (WAV, FLAC, OGG)
NumPy	≥ 1.24.0	Numerical computation and array operations

9.3 API & Serving

Technology	Version	Purpose
FastAPI	≥ 0.110.0	Async REST API framework with auto-generated docs
Uvicorn	≥ 0.29.0	High-performance ASGI server
python-multipart	≥ 0.0.9	File upload parsing for FastAPI
Pydantic v2	(via FastAPI)	Type-safe response schema validation

9.4 Production Hardening (Second Semester)

Technology	Version	Purpose
slowapi	≥ 0.1.9	Rate limiting middleware
structlog	≥ 24.1.0	Structured logging (JSON and console)
python-json-logger	≥ 2.0.7	JSON log formatting support

9.5 Pre-trained Model

Model	Source	Parameters	Pre-training
facebook/hubert-base-ls960	Hugging Face Hub	~95M	Self-supervised on LibriSpeech 960h

10. Performance Analysis

10.1 Stage 1 — Binary Cry Detection

Metric	Value
Test Accuracy	99.42%
Macro F1-Score	0.9937
ROC-AUC	0.9983
baby_cry F1	0.9955
not_baby_cry F1	0.9920
False Positives	4 / 560 (0.71%)
False Negatives	5 / 992 (0.50%)

Stage 1 Confusion Matrix:

		Predicted	
		baby_cry	not_baby_cry
True	baby_cry	987	5
	not_baby_cry	4	556

Total: 1,552 test samples — only 9 misclassifications.

Analysis: Stage 1 achieves near-perfect detection. The 0.50% false negative rate means approximately 1 in 200 genuine cry events would be missed — acceptable for an advisory system. The 0.71% false positive rate is similarly low, meaning non-cry audio rarely triggers the Stage 2 pipeline unnecessarily.

10.2 Stage 2 — 4-Class Cry Classification

Metric	Value
Test Accuracy	84.27%
Macro F1-Score	0.7685
Best Validation F1	0.7894
Wrong-confident predictions	11.5%

Per-Class Performance:

Class	F1-Score	Recall	Assessment
scared	0.967	99%	Excellent — distinct acoustic signature
needs	0.881	90%	Strong — benefits from label merging
physical_pain	0.687	70%	Moderate — confused with needs (29%)
burping	0.538	41%	Weak — severe data scarcity

Stage 2 Confusion Matrix (Counts):

		Predicted			
		burping	needs	physical_pain	scared
True	burping	28	37	2	2
	needs	6	561	54	5
	physical_pain	1	47	114	0
	scared	0	2	0	133

Stage 2 Confusion Matrix (Row-Normalized):

		Predicted			
		burping	needs	physical_pain	scared
True	burping	0.41	0.54	0.03	0.03
	needs	0.01	0.90	0.09	0.01
	physical_pain	0.01	0.29	0.70	0.00
	scared	0.00	0.01	0.00	0.99

Key Observations:

- **Burping → Needs confusion (54%):** Burping cries are short and low-intensity, acoustically similar to mild fussing.
- **Physical_pain → Needs confusion (29%):** Pain cries can overlap with intense needs-based fussing.
- **Scared is highly distinctive:** Sharp onset, high-pitch — easy for the model to isolate (99% recall).

10.3 Inference Performance

Setup	RAM	GPU VRAM	Inference Time
GPU (T4 / RTX 3060+)	4 GB	4 GB	~150 ms
CPU only	4 GB	—	~2 seconds

10.4 Model Storage

Checkpoint	Size
stage1_binary.ckpt	1.03 GB
stage2_4class_epoch6.ckpt	1.03 GB
stage2_4class_epoch7.ckpt	1.03 GB
Total	~3.30 GB

11. Security & Reliability

11.1 API Protection

Concern	Implementation
CORS	allow_origins=["*"] (permissive for development; tighten for production)
Rate Limiting	30 requests/minute, 500/hour per IP via slowapi
Request Tracing	UUID per request in logs and responses
HTTPS	Not configured (to be terminated at reverse proxy in production)

11.2 Input Validation (Multi-Layer)

The system implements a three-layer defense against bad inputs:

Layer	Module	What It Catches
Layer 1: File Validation	validators.py	Wrong format, corrupt files, oversized uploads, empty files, too-short/long audio
Layer 2: Acoustic Checks	acoustic_checks.py	Silence, noise, constant tones, wrong frequency profile
Layer 3: Model Inference	inference.py	Misclassification mitigated by ensemble averaging and confidence thresholds

11.3 Model Safety

- **No data persistence:** Audio is processed in-memory and discarded — no server-side storage.
- **Medical disclaimer:** Always included in every response, regardless of prediction outcome.
- **Confidence transparency:** Low-confidence predictions trigger explicit Arabic advisory text (e.g., "التطبيق غير متأكد تماماً من النوع").
- **Not a diagnostic tool:** Clearly documented as AI assistance, not medical diagnosis.

11.4 Error Handling

Error Scenario	Status Code	Handling
Invalid file format/size	400	Bilingual error with specific error code
Acoustic check failure	400	Bilingual error with signal measurements
Audio decode failure	400	Bilingual error with exception details
Rate limit exceeded	429	Bilingual error with Retry-After header
Inference failure	500	Bilingual error with exception type
Models not loaded	503	Service Unavailable with Arabic message

11.5 Reliability

- **Stateless design:** Each request is independent — no session state, no inter-request dependencies.
- **Graceful startup:** Health endpoint reports `models_loaded: false` until initialization completes.
- **Deterministic inference:** `@torch.no_grad() + model.eval()` ensures consistent outputs.
- **Model warm-up:** Dummy inference on startup ensures the first real request has normal latency.

12. Deployment & Execution

12.1 Local Development

```
# 1. Clone repository
git clone <repository-url>
cd BabyCry

# 2. Create virtual environment (Python 3.10+ recommended)
python -m venv venv
venv\Scripts\activate          # Windows
# source venv/bin/activate     # Linux/macOS

# 3. Install dependencies
pip install -r requirements.txt

# 4. Start the API server (development mode)
python run_api.py
# Server starts at http://127.0.0.1:8000
# Swagger docs at http://127.0.0.1:8000/docs
# Health check at http://127.0.0.1:8000/health

# 5. Start in production mode
set ENV=production && python run_api.py  # Windows
# ENV=production python run_api.py      # Linux/macOS
```

12.2 Environment Requirements

Requirement	Minimum	Recommended
Python	3.10	3.11+
RAM	4 GB	8 GB
GPU VRAM	4 GB (optional)	6 GB+
Disk	5 GB (models + cache)	10 GB
OS	Windows / Linux / macOS	Linux (for production)

12.3 Server Startup Process

```
run_api.py
|
| uvicorn.run("api.main:app")
▼
Uvicorn (ASGI Server)
|
| Initialize FastAPI app
▼
FastAPI App
|
| @app.on_event("startup")
▼
load_model()
|
|─ Check CUDA availability
|─ Download HuBERT base (first run only, cached after)
|─ Load stage1_binary.ckpt (~1 GB)
|─ Load stage2_4class_epoch6.ckpt (~1 GB)
|─ Load stage2_4class_epoch7.ckpt (~1 GB)
|─ Run warm-up inference (5s silence)
|
▼
Server Ready
"Uvicorn running on http://127.0.0.1:8000"
```

12.4 CLI Inference (Without API)

For direct inference without starting the API server:

```
python inference/inference.py path/to/audio.wav
```

12.5 Production Deployment Options

Method	Complexity	Suitable For
Uvicorn + Nginx	Low	Single-server deployment
Docker	Medium	Containerized deployment
Docker Compose	Medium	Multi-service orchestration
Google Cloud Run	Medium	Managed container platform
Kubernetes	High	Scalable cloud deployment

13. Arabic Localization

The module provides full Arabic localization for all user-facing content. All Arabic text is centralized in `api/content.py`, completely decoupled from the business logic.

13.1 Cry Type Content

Each cry type includes the following Arabic content:

Field	Description	Example (needs)
emoji	Visual identifier	👶
name_ar	Arabic name	احتياج عام
definition_ar	Definition of the cry type	بكاء عام يدل على حاجة أساسية: جوع، تعب، أو عدم راحة
physical_signs_ar	List of physical signs to observe	جوع: عادة بعد 2-4 ساعات من الرضعة الأخيرة
tips_ar	Actionable parenting tips	افحصي الحفاضة أولاً
warning_ar	When to consult a doctor	راقبي إشارات طفلك قبل البكاء

13.2 Confidence Level Labels

Confidence Range	Arabic Label	English Equivalent
≥ 0.85	ثقة عالية جداً	Very high confidence
0.70 – 0.84	ثقة عالية	High confidence
0.55 – 0.69	ثقة متوسطة	Medium confidence
0.40 – 0.54	ثقة منخفضة	Low confidence
< 0.40	غير متأكد	Uncertain

When confidence falls below 0.55, an additional Arabic advisory is included:

التطبيق غير متأكد تماماً من النوع. قد يكون من الأفضل تجربة تسجيل آخر أو الاعتماد على ملاحظتك المباشرة لطفلك 🚩

13.3 Medical Disclaimer

A medical disclaimer is always included in every response, regardless of the prediction:

تنبيه مهم: هذا التطبيق يستخدم الذكاء الاصطناعي لمساعدتك في فهم بكاء طفلك، ولكنه ليس بديلاً عن الاستشارة الطبية.

14. Challenges & Solutions

14.1 Audio Preprocessing Variability

Problem	Audio arrives in diverse formats (WAV, MP3, OGG) with varying sample rates (8kHz–48kHz), channel counts, and bit depths.
Solution	Universal <code>preprocess_audio()</code> function with auto-format detection, stereo-to-mono averaging, <code>librosa.resample()</code> to 16kHz, and int16 auto-scaling. In the second semester, a validation layer was added to reject unsupported formats before processing.

14.2 Class Imbalance

Problem	<code>burping</code> has only ~240 training samples vs. thousands for <code>needs</code> — standard training converges to always predicting <code>needs</code> .
Solution	Inverse-frequency class weighting (<code>burping</code> weight=2.07, <code>needs</code> weight=0.20) in the cross-entropy loss, ensuring minority-class gradients are amplified.

14.3 Acoustic Similarity Between Classes

Problem	<code>physical_pain</code> and <code>needs</code> overlap significantly in spectral characteristics — 29% of pain samples are misclassified as <code>needs</code> .
Solution	Label smoothing ($\epsilon=0.1$) prevents overconfident predictions; ensemble averaging reduces variance; confidence-aware advisories warn users when classification is uncertain.

14.4 Variable-Length Audio

Problem	Audio clips range from less than 1 second to more than 30 seconds, but HuBERT requires fixed-length input.
Solution	Center-cropping (for long audio) and zero-padding with attention masking (for short audio). The masked mean pooling in the classification head ensures padded regions do not influence predictions.

14.5 Model Size and Loading Latency

Problem	Three HuBERT checkpoints total ~3.3 GB — loading on every request would be prohibitively slow.
Solution	Singleton pattern: load all models once during startup and reuse the shared instance for all subsequent requests. Additionally, model warm-up on startup ensures the first real request has normal latency.

14.6 Source Domain Shift

Problem	Models trained primarily on one dataset source show degraded performance on recordings from different microphones/environments (F1 drops from 0.78 to 0.38 across sources).
Solution	Multi-source training data aggregation. This remains a known limitation with a documented expected accuracy drop of 10–20 percentage points on out-of-domain recordings.

14.7 Malicious or Irrelevant Inputs (Second Semester)

Problem	In a production environment, the API may receive non-audio files, silent recordings, constant tones, or white noise — which could waste GPU compute or produce meaningless predictions.
Solution	Multi-layer input defense: file validation (format, size, duration) followed by acoustic sanity checks (RMS, voiced ratio, frequency band, spectral flux, ZCR). Both layers return clear bilingual error messages.

14.8 API Abuse Prevention (Second Semester)

Problem	Without rate limiting, a single client could overwhelm the GPU with rapid-fire requests, degrading service for other users.
Solution	Per-IP rate limiting at 30 requests/minute and 500 requests/hour using <code>slowapi</code> , with a clear <code>429 Too Many Requests</code> response and <code>Retry-After</code> header.

15. Known Limitations

- 1. **Source Bias:** Training data is heavily weighted toward one source (`baby_crying` dataset). Real-world recordings from different microphones or environments may experience a 10–20 percentage point accuracy drop.
- 2. **Burping Class:** Only ~240 training samples – model recall is moderate (41%). This class is frequently confused with `needs` .
- 3. **Not a Diagnostic Tool:** The system is designed for AI assistance only and should not be used as a substitute for medical evaluation.
- 4. **Memory Usage:** The three HuBERT checkpoints require ~5 GB of RAM/VRAM per server instance.
- 5. **Single-Process Serving:** The current architecture serves requests in a single process, limiting concurrency under heavy load.

16. Future Improvements

16.1 Short-Term

Feature	Description	Expected Impact
ONNX Export	Convert HuBERT models to ONNX format	2–5× CPU inference speedup
Backbone Sharing	Share a single HuBERT backbone between Stage 1 and Stage 2	~60% memory reduction
Docker Containerization	Package the system with pre-cached weights	Simplified deployment
API Authentication	Add API key or JWT-based authentication	Production security

16.2 Medium-Term

Feature	Description	Expected Impact
Real-Time Streaming	WebSocket endpoint for continuous audio monitoring	Smart nursery applications
Mobile SDK	Native iOS/Android SDK wrapping the REST API	Better developer experience
Multilingual Support	Extend content to English, French, Urdu	Broader market reach
Data Augmentation	SpecAugment, noise injection, pitch shifting	Improved model robustness

16.3 Long-Term

Feature	Description	Expected Impact
Edge AI Deployment	TensorFlow Lite or CoreML for on-device inference	Offline capability, privacy
Explainable AI (XAI)	Attention heatmaps showing which audio segments drove the prediction	Trust and transparency
Multi-infant Tracking	Speaker diarization to distinguish cries from multiple infants	NICU applications
Emotion Intensity Scoring	Regression output for cry intensity (mild → severe)	Triage prioritization

17. Conclusion

The Baby Cry Recognition Mode represents a sophisticated application of self-supervised speech representation learning to the pediatric healthcare domain. By leveraging HuBERT — a model pre-trained on 960 hours of adult English speech — and fine-tuning it on infant cry recordings, the module demonstrates that modern speech foundation models capture universal acoustic features that transfer effectively across vocal domains, age groups, and linguistic contexts.

The two-stage hierarchical cascade architecture is an efficient design decision: Stage 1 achieves near-perfect detection accuracy (99.42%), acting as a high-precision gatekeeper that prevents false alarms and unnecessary Stage 2 computation. Stage 2's ensemble approach — averaging softmax probabilities across two checkpoints from different training epochs — provides a principled mechanism to reduce prediction variance.

The second semester's production hardening phase transformed the system from a research prototype into a deployment-ready service. The addition of multi-layer input validation, acoustic sanity checks, rate limiting, structured logging, and metrics tracking ensures the system is robust against real-world conditions — malformed inputs, abusive traffic patterns, and operational monitoring requirements.

The Arabic-first localization — including cry type definitions, physical signs, actionable parenting tips, and medical warnings — demonstrates a user-centered design philosophy that goes beyond model accuracy to deliver meaningful human impact for Arabic-speaking caregivers.

Appendix

A. Pydantic Response Schemas

The API responses are defined using Pydantic v2 models for type safety and automatic validation:

Schema	Fields
PredictionResponse	success, request_id, audio_info, acoustic_check, stage1, stage2, medical_disclaimer_ar, processing_time_ms
Stage1Result	is_baby_cry, emoji, title_ar, description_ar, confidence, confidence_label_ar
Stage2Result	type, emoji, name_ar, definition_ar, physical_signs_ar, tips_ar, warning_ar, confidence, confidence_label_ar, confidence_advice_ar, all_probabilities
AudiInfo	duration_seconds, sample_rate, channels, format, size_kb
ErrorResponse	success, error_code, error_message_ar, error_message_en, request_id, measurements
HealthResponse	status, models_loaded, device, version, metrics

B. Acoustic Sanity Check Thresholds

Metric	Threshold	Purpose
RMS Energy	≥ 0.010	Minimum loudness (~40 dBFS)
Voiced Ratio	≥ 0.20	At least 20% of frames must be voiced
Cry Band Ratio (250–3000 Hz)	≥ 0.40	At least 40% of energy in cry frequency band
Spectral Flux	≥ 0.015	Minimum temporal variation
Zero-Crossing Rate	0.02 – 0.35	Expected range for vocal sounds

C. Stage 2 Label Merge Map

Original Label	Merged Label	Rationale
hungry	needs	Similar acoustic profile; shared caregiving response
tired	needs	Similar acoustic profile; shared caregiving response
discomfort	needs	Similar acoustic profile; shared caregiving response

belly_pain Original Label	physical_pain Merged Label	Renamed for clinical clarity Rationale
scared	scared	Preserved – acoustically distinctive
burping	burping	Preserved – distinct temporal pattern

D. Dataset Source Licenses

Dataset	License
ESC-50	CC BY-NC 4.0
infant_cry_corpus (Donate-a-Cry)	CC BY-NC 4.0
baby_crying (Kaggle)	See source
baby_sounds (Kaggle)	See source

E. Key File Reference

File	Lines	Size	Purpose
inference/inference.py	250	9.5 KB	Core AI module: model class, preprocessing, inference orchestrator
api/main.py	354	12.7 KB	FastAPI app: endpoints, model lifecycle, response assembly
api/content.py	175	7.8 KB	Arabic localization: cry descriptions, tips, warnings
api/schemas.py	80	2.9 KB	Pydantic models: response schemas
api/validators.py	122	4.8 KB	Audio file validation (size, format, duration)
api/acoustic_checks.py	178	6.8 KB	Acoustic sanity checks (RMS, ZCR, frequency)
api/rate_limit.py	30	1.0 KB	Rate limiting configuration
api/logging_config.py	54	1.6 KB	Structured logging setup
api/metrics.py	56	2.0 KB	In-memory metrics tracking
config/training_config.json	44	836 B	Training hyperparameters and class weights
run_api.py	31	800 B	Uvicorn server launcher
requirements.txt	19	492 B	Python dependencies